

从“答案正确”到“结论可审计”： 面向 AI4Math 的 TRACE 分层可靠性框架

匿名作者

独立研究者（匿名评审稿）

2026 年 9 月

摘要

大语言模型在数学解题、定理证明与数学发现中表现出快速提升的能力，但“最终答案正确”不足以推出“推理过程可靠”：生成器可能偶然命中答案，学习型验证器可能与生成器共享偏差，形式化证明也可能证明了一个与原题不等价的命题，而公开分数可能因数据污染、预算不透明或环境漂移而无法复现。本文提出 TRACE (Task/Data Traceability、Reasoning Search、semantic Alignment、Certification、Evaluation Replay) 五门可靠性框架，将 AI4Math 系统的可信性拆解为任务与数据溯源、候选搜索与过程判别、语义规约对齐、确定性确证以及可复现实验五个连续门控。本文的核心观点是：形式化内核只能保证“给定形式化陈述下的证明有效”，不能自动保证自然语言问题与形式化陈述等价；因此，语义规约层是自然语言数学走向机器确证时的关键瓶颈。我们进一步给出门控式选择性发布规则、基于并集界的残余错误上界、风险-覆盖率与复现率等指标，并对 18 项代表性一手研究进行结构化证据编码。分析显示：搜索与验证能提高候选命中率，却不能把学习型评分器变成真值判定器；形式化证明提供最强的局部正确性保证，却把风险前移到陈述翻译、库版本和规格覆盖；公开基准改善可比较性，但仍需污染审计、预算披露和环境锁定。本文不报告新的模型性能，而贡献一个可证伪、可审计、可复用的系统评价框架及最小复现清单，为研究论文、模型评测和高风险数学应用提供共同的可靠性语言。

关键词： AI for Mathematics；数学推理；过程验证；自动形式化；定理证明；可复现评估；选择性预测

English Title: From Correct Answers to Auditable Claims: The TRACE Layered Reliability Framework for AI4Math

1 引言

数学推理具有一个看似有利、实则容易被误解的特征：最终答案常常可被简短地判定。然而，答案级判定只观察输出末端，不能说明中间论证是否有效，更不能说明系统在题意理解、形式化翻译、工具调用与实验报告中是否保持了同一语义。以 GSM8K 为代表的工作展示了“多候选生成 + 验证器重排”的有效性 [1]，MATH 则把评测推进到更复杂的竞赛题 [2]；过程监督研究进一步表明，结果监督会把“过程错误但答案偶合正确”的轨迹视为正例 [3, 4]。另一方面，Lean 等证

明助理提供小型可信内核，使形式化证明可以被机器确定性检查 [5, 6]。这两条路线分别缓解“找不到正确候选”和“无法严格检查候选”的问题，却没有自动解决自然语言题意如何被无损映射到可检查对象的问题。

这一断点可概括为“验证对象缺口”：验证器验证的对象未必就是用户原本提出的命题。自然语言中的量词范围、隐含非退化条件、数域、单位、近似规则与图像约束，都可能在程序化或形式化时被省略或改变。自动形式化已显示出可行性 [7, 8]，但其成功率和语义保真并非同一个指标。即使形式化证明完全通过，系统也可能只是严格证明了一个错误翻译后的命题。

本文研究三个问题：

RQ1: 如何把“可靠”从单一正确率分解为可定位、可检验的系统属性？

RQ2: 如何连接概率式生成、学习型验证与形式化确证，同时避免把任一组件误当成全局正确性保证？

RQ3: 什么样的报告协议能够让第三方复查某项 AI4Math 结论的来源、语义、证明与实验环境？

本文作出三项贡献。第一，提出 TRACE 五门框架及跨层失效模型，将可靠性从模型属性改写为端到端证据链属性。第二，提出门控式选择性发布规则与残余错误并集界，明确系统应何时拒答，以及任何“可靠性分数”成立所需的假设。第三，对 18 项代表性一手研究进行机制编码，并给出可直接用于论文与基准报告的“可靠性卡”和最小复现清单。本文是一篇框架与证据综合论文，不声称进行了新的模型训练或性能对比。

2 概念界定与研究方法

2.1 可靠性不是准确率的同义词

给定自然语言任务 x 、系统输出 y 与目标语义 $\mathcal{S}(x)$ ，本文把可靠输出定义为：

$$\text{Reliable}(x, y) = \text{Faithful}(x, z) \wedge \text{Valid}(z, \pi) \wedge \text{Grounded}(y; z, \pi) \wedge \text{Replayable}(m), \quad (1)$$

其中 z 是结构化或形式化的中间表示， π 是可检查证据（证明对象、程序运行轨迹、约束证书或逐步判定记录）， m 是包含数据、模型、预算、版本与随机性的实验清单。式 (1) 区分四类常被混淆的事实：答案可能正确但过程无效；证明对象可能有效但陈述失真；自然语言解释可能与已验证对象不一致；一次运行可能成功但不可复现。

2.2 文献选择与证据编码

本文采用面向机制的结构化证据综合，而非声称穷尽全部 AI4Math 文献。纳入标准为：(1) 直接提出数学生成、验证、形式化或评测方法；(2) 有可核验的一手论文页、DOI 或 arXiv 标识；(3) 能够回答至少一个研究问题。核心样本覆盖 2020–2026 年的 18 项研究，包括结果/过程验证 [1, 3, 4, 9]、数学语言模型与公开基准 [2, 10–12]、形式化与自动形式化 [5–8, 13–16]，以及神经符号发现与证明系统 [17–19]。

每项研究按 TRACE 五门是否被显式实现或报告编码为“覆盖”“部分覆盖”“未覆盖/不适用”。该编码评价的是证据链完备性，不是模型能力排名；“未覆盖”不等于工作质量低，而意味着该门不在原研究目标内。文献元数据以出版商、会议官方页面、Crossref 或 arXiv 记录交叉核

验。为避免把综述性判断伪装成实验结果，本文不从图表反推未报告数字，不跨论文直接比较不同模型、提示、采样预算或数据切分下的准确率。

3 相关工作

3.1 搜索与学习型验证

生成器一次采样的失败可通过扩大测试时搜索得到缓解。给定候选集合 $C(x) = \{c_1, \dots, c_k\}$ ，结果验证器通常选择

$$c^* = \arg \max_{c \in C(x)} V_{\text{out}}(x, c), \quad (2)$$

其中 V_{out} 只依据最终答案或完整解答给分。该范式在数学文字题上建立了“生成-验证”基线 [1]。但当错误推理偶然得到正确答案时，结果监督会提供错误的正反馈。过程奖励模型把轨迹 $c = (s_1, \dots, s_T)$ 分解并评估中间步骤 [3, 4]；Math-Shepherd 则探索无需逐步人工标注的过程监督 [9]。

这些方法的共同限制是：学习型验证器仍是统计模型。生成器与验证器可能继承相同训练分布、表述偏差或数据污染，增加采样预算还可能把搜索推向“最能欺骗评分器”而非最正确的候选。因此，学习型验证适合作为搜索启发式和风险筛选器，而不应被表述为形式化意义上的正确性证明。

3.2 形式化证明与语义规约

Lean 4 以小型内核检查证明对象 [5]，mathlib 提供可复用的形式化数学基础设施 [6]。miniF2F 使奥数级形式化证明可以跨系统比较 [13]，LeanDojo 则把环境交互、检索、数据切分和开源工具链纳入可复现实验 [15]。非正式证明草图可用于引导形式证明搜索 [14]，ProofNet 将问题扩展到本科数学的“自然语言到形式化再到证明”链路 [8]。

形式化路线提供的是相对于形式陈述和内核的可靠性。自动形式化工作显示，大语言模型能够把部分自然语言题目转换为形式规范 [7]，但“语法可编译”“定理可证明”与“语义等价”并不相同。若形式化陈述漏掉约束，内核会忠实地确认错误规格。AlphaGeometry 采用专用几何语言与符号引擎，说明缩窄表示域能够提高可检查性 [17]；其边界同样表明，表示覆盖率是可靠性的一部分。

3.3 可复现评估

MATH 提供分学科、分难度的竞赛题及解答 [2]，OlympiadBench 增加双语、多模态和过程级评分需求 [11]，FrontierMath 通过保密题目降低污染风险，但集中式评测也限制了第三方本地复现 [12]。这些工作体现了两种张力：公开数据便于复现却更易污染；保密数据有助于测量未见题能力，却降低审计透明度。因而，可靠评估不能只报告一个准确率，还应披露数据暴露风险、采样预算、判分器、工具版本和失败样本。

4 TRACE: 五门分层可靠性框架

4.1 总体结构

TRACE 把端到端系统看作一条带拒答机制的证据流水线，如图 1 所示。每一门都回答不同问题，任一门失败都应阻止“已验证”标签继续传播。

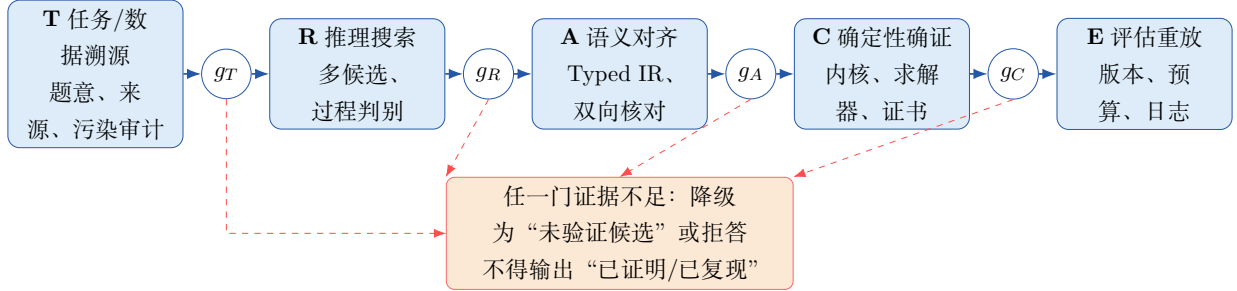


图 1: TRACE 五门可靠性框架。圆形门控表示必须记录的验收判据，而非新的生成模块。

4.2 T: 任务与数据溯源门

第一门固定“正在回答什么”。系统应保存原题、图像/附件、语言版本、时间戳与内容哈希；显式列出数域、量词、单位、非退化条件和允许使用的外部定理。评测时还需记录题目是否公开、是否可能进入预训练或微调数据，以及测试集是否在开发期间被查看。该门的主要失效是任务漂移与数据污染：系统对一个更容易的改写题给出正确答案，或在记忆题目上获得高分，却被误解为一般推理能力。

4.3 R: 推理搜索与过程判别门

第二门负责提高“找到可验证候选”的概率，而非授予最终正确性。候选应保留采样温度、随机种子、最大 token、工具调用与搜索预算；过程验证器输出步骤分数和失败位置。对于 best-of- k ，至少同时报告 k 、生成成本和候选覆盖率，防止把计算预算差异解释为算法差异。过程监督相较结果监督提供更细粒度信号 [4]，但自动构造标签仍可能把生成器偏差带入验证器 [9]。

4.4 A: 语义对齐与规约门

第三门把自然语言 x 和候选推理 c^* 映射为类型化中间表示 $z = A(x, c^*)$ 。 z 可以是 Lean/Coq 陈述、符号程序、约束系统、抽象语法树或带类型的证明图。与只要求“能编译”不同，TRACE 要求双向对齐：由 z 反向生成规范化自然语言 $\hat{x} = D(z)$ ，并检查

$$\text{Align}(x, z) = \text{Entails}(x, \hat{x}) \wedge \text{Entails}(\hat{x}, x) \wedge \text{ConstraintsEqual}(x, z). \quad (3)$$

其中蕴含检查可由领域规则、对抗性测试和人工审阅共同完成。人类签核必须面对“原题–规范题意–形式陈述”三列对照，而不是只看可读性更高的自然语言解释。该门是全文强调的语义规约瓶颈：它位于概率生成与确定性验证之间，既不能由语言模型自我声明完成，也不能由证明内核代替。

4.5 C: 确定性确证门

第四门要求使用与任务相匹配的独立检查器：形式证明由可信内核验证，代数恒等式由计算机代数系统与精确算术检查，组合构造由可执行判定器验证，数值结论则需要误差界或区间算术。FunSearch 将 LLM 的候选生成与可执行评价函数配对，展示了“难生成、易验证”问题上的有效闭环 [18]；AlphaGeometry 通过神经辅助构造与符号演绎结合提供了更强的几何确证 [17]；AlphaProof 则进一步展示了强化学习与 Lean 检查的结合 [19]。但确证门的结论严格限定为：给定 z 、库 L 、内核 K 与假设集 H ，证据 π 被接受。它不自动覆盖 $x \leftrightarrow z$ 的语义等价、库公理选择或实现缺陷。

4.6 E: 评估重放门

第五门要求第三方能够在隔离环境重放结论。最小清单包括：数据快照及哈希、训练/验证/测试切分、模型与权重版本、提示模板、解码参数、候选数与总算力预算、验证器版本、定理库与证明器版本、评分脚本、随机种子、硬件/软件环境、失败样本与人工裁决规则。LeanDojo 对环境、数据和模型的开源整合说明，形式化证明也需要工程层面的复现基础设施 [15]。对于闭源模型或保密题库，应把“可重复获得相近统计结果”与“可逐项重放相同轨迹”分开报告。

4.7 门控发布与残余风险上界

定义发布变量 $G = \prod_{i \in \{T, R, A, C, E\}} g_i$ ，其中 $g_i \in \{0, 1\}$ 表示第 i 门证据是否满足预注册阈值。只有 $G = 1$ 时，系统才能把结果标记为“经完整证据链验证”；否则应输出“未验证候选”或拒答。为了避免单一总分掩盖短板，建议报告可靠性向量

$$\mathbf{q} = (q_T, q_R, q_A, q_C, q_E), \quad q_i \in [0, 1], \quad (4)$$

而不把它任意加权成一个看似精确的标量。

命题 1 (门控链的保守错误界). 设 F_i 表示第 i 门发生未被发现的失效，并假设任何已发布的错误结论至少伴随一个 F_i 。若在目标分布上 $\Pr(F_i) \leq \epsilon_i$ ，则

$$\Pr(\text{错误且发布}) \leq \sum_i \epsilon_i. \quad (5)$$

若覆盖率 $\kappa = \Pr(G = 1) > 0$ ，则

$$\Pr(\text{错误} \mid G = 1) \leq \min \left(1, \frac{\sum_i \epsilon_i}{\kappa} \right). \quad (6)$$

证明. 由前提，事件“错误且发布”包含于 $\bigcup_i F_i$ 。应用 Boole 不等式得到第一式；再除以 $\Pr(G = 1) = \kappa$ 并用概率上界 1 截断，得到式 (6)。□

该命题的价值在于暴露假设，而非宣称现实系统已有可认证概率。若 ϵ_i 没有独立标注集、对抗测试或形式化保证支撑，就不能把式 (6) 当作数值证书。门控也必然牺牲覆盖率，因此可靠性评估必须同时报告风险与覆盖率。

5 代表性研究的跨层证据分析

表 1 对代表性工作进行机制级编码。符号 Y 表示论文明确实现或提供该门的主要证据，P 表示仅部分覆盖，- 表示未覆盖或不适用。表格不是性能排行榜。

表 1: 代表性研究对 TRACE 五门的覆盖（基于原论文机制描述；Y= 覆盖，P= 部分覆盖，-= 未覆盖或不适用）

工作	T	R	A	C	E	主要残余风险
Verifier/GSM8K[1]	P	Y	-	P	P	结果正确无法排除过程错误；公开题污染
过程反馈 [3]	P	Y	-	P	P	步骤标签与任务外泛化仍有限
PRM800K[4]	P	Y	-	P	P	学习型 PRM 不是逻辑判定器，标注成本高
Math-Shepherd[9]	P	Y	-	P	P	自动标签可能继承 rollout 模型偏差
MATH[2]	Y	-	-	P	Y	最终答案评分弱化过程可靠性；污染风险
OlympiadBench[11]	Y	-	P	P	Y	多模态与过程评分需严格复现裁决协议
FrontierMath[12]	Y	-	-	P	P	保密题抗污染，但独立本地重放受限
miniF2F[13]	Y	P	P	Y	Y	形式陈述与自然语言题意的等价性需另审
自动形式化 [7]	P	P	Y	Y	P	可证明的翻译未必语义等价
LeanDojo[15]	Y	Y	P	Y	Y	Lean/mathlib 版本与数据切分影响复现
ProofNet[8]	Y	P	Y	Y	P	本科题覆盖有限；需人工语义校对
AlphaGeometry[17]	P	Y	Y	Y	P	专用表示覆盖范围窄，系统复现门槛高
FunSearch[18]	P	Y	Y	Y	P	仅适用于存在强可执行评价函数的问题
AlphaProof[19]	P	Y	Y	Y	P	形式化陈述、算力与闭源组件限制复现

5.1 发现一：搜索扩大“找到正确候选”的机会，不等于建立真值判定

从结果验证器到过程奖励模型，研究重点逐渐从“答案是否匹配”转向“每一步是否可信”[1, 4]。这是重要进步，但验证器与生成器同属学习系统时，二者错误并非独立：相同语料、相似架构与相同表述习惯会造成相关失效。因而，best-of- k 的收益应与验证器的域外校准、对抗错误集表现和搜索预算同时报告。只报告最终准确率，无法区分“搜索覆盖提高”与“验证准确性提高”。

5.2 发现二：形式化确证把风险从证明步骤前移到规格

证明内核能排除非法推理步骤，却不能判断形式陈述是不是原题。自动形式化的进展 [7] 和自然语言草图引导证明 [14] 说明，语义规约不是一次性翻译，而是需要反向解释、约束测试与人工签核的独立环节。本文因此把 A 门与 C 门拆开：A 门问“证明的是不是同一件事”，C 门问“这个形式对象在指定系统内是否成立”。把二者合并会产生最危险的假阳性：机器给出完全通过的证明，但回答了错误问题。

5.3 发现三：可复现性与抗污染性必须分别测量

公开基准支持脚本复跑和横向比较，却可能被训练语料污染；保密基准减少记忆命中，却让题目、评分器与失败样本难以独立审计。两者不是简单的优劣关系。应分别报告 (1) **重放透明度**：

第三方能否在给定环境复现流程；(2) **新颖性可信度**：测试样本进入训练数据的风险；(3) **裁决独立性**：评分器是否独立于被评模型。把这三项压缩成“benchmark score”会隐藏重要差异。

6 可复现实验与审计协议

6.1 最小实验设计

若后续把 TRACE 用于实证评估，建议采用分层、配对设计，而不是比较来源不同的排行榜数字。

1. **任务分层**：至少包含答案型文字题、自然语言证明题、已形式化定理三类；按难度和领域分层抽样。
2. **四个消融系统**：单次生成；best-of- k + 结果验证；best-of- k + 过程验证；完整 TRACE（加入语义规约与确定性检查）。所有系统使用相同生成模型、提示和总 token 预算。
3. **双盲语义审计**：两名审阅者独立标注形式陈述为“等价/偏强/偏弱/不相关”，分歧由第三名审阅者裁决，并报告一致性系数。
4. **环境锁定**：保存容器镜像摘要、依赖锁文件、证明器与定理库提交号、评分脚本哈希、随机种子和失败日志。
5. **预注册**：在查看测试结果前固定主要指标、拒答阈值、统计检验与排除规则；不得依据结果修改口径。

6.2 指标组

可靠性应以向量而非单一准确率报告：

$$\text{AnsAcc} = \frac{\text{\#最终答案正确}}{N}, \quad (7)$$

$$\text{ProcPass} = \frac{\text{\#无已知无效步骤的轨迹}}{N}, \quad (8)$$

$$\text{SemEq} = \frac{\text{\#经裁决语义等价的形式化陈述}}{\text{\#成功形式化}}, \quad (9)$$

$$\text{CertRate} = \frac{\text{\#检查器接受且语义等价}}{N}, \quad (10)$$

$$\text{ReplayRate} = \frac{\text{\#干净环境中成功重放}}{\text{\#重放尝试}}. \quad (11)$$

对于带拒答的系统，阈值 τ 下还应报告

$$\text{Risk}(\tau) = \frac{\text{\#已接受但错误}}{\text{\#已接受}}, \quad \text{Coverage}(\tau) = \frac{\text{\#已接受}}{N}. \quad (12)$$

只报告低风险而不报告覆盖率，会奖励“几乎全部拒答”的无用系统。成本维度至少包括每题候选数、总 token、工具调用次数、证明器 CPU 时间和人工审阅分钟数。

6.3 可靠性卡

每个公开结论应附一张机器可读或表格化的可靠性卡：

表 2: TRACE 可靠性卡的最小字段

门	必填证据	失败时的发布标签
T	原题/附件哈希、题意假设、数据来源、污染风险	任务来源不明
R	模型与提示版本、随机种子、 k 、预算、候选与过程分数	未验证候选
A	Typed IR、反向释义、约束测试、人工语义签核	形式化未对齐
C	证明对象/程序证书、内核或求解器版本、检查日志	未确证
E	环境锁、评分脚本、完整配置、失败样本、重放记录	不可复现

7 讨论

7.1 框架的适用方式

TRACE 不是要求所有数学任务都立即形式化。对于小学算术，可执行程序 and 精确答案检查可能已足够；对于自然语言证明，应增加步骤级审阅与反例搜索；对于高风险定理声明，只有语义对齐后的形式证明或等价强度的独立证据才应获得“已确证”标签。关键不是一刀切地追求最重工具，而是让证据强度与结论风险相匹配。

该框架也解释了两类成功范式。Minerva 等模型说明领域数据与规模能显著提升定量推理能力 [10]，但能力提升本身不是可靠性证明。相反，AlphaGeometry、FunSearch 与 AlphaProof 的共同结构是把开放式生成限制在可被符号规则、执行器或证明内核检查的候选空间 [17–19]。这类“生成负责提出，检查器负责否决”的分工比模型自我确认更稳健，但仍需 A 门与 E 门防止规格错位和不可复现。

7.2 面向研究者与评测者的建议

- 把“证明通过”写成“在版本化形式陈述与指定内核下通过”，不要省略相对性。
- 把学习型验证器定位为风险模型，报告其校准、域外测试和与生成器共享数据的情况。
- 同时公布接受错误率与覆盖率；把拒答作为可靠系统的正常输出，而不是失败样本隐藏机制。
- 对自动形式化进行双向语义审计，尤其检查量词、数域、单位、边界与非退化条件。
- 把复现包视为论文论证的一部分，而非附录装饰；版本、预算和失败轨迹必须可追踪。

8 局限性与效度威胁

第一，本文是机制型框架和结构化证据综合，不是系统综述；文献样本具有目的性，可能遗漏其他证明助理、非英语工作和未公开工业系统。第二，表 1 的门覆盖编码只依据公开论文描述，

不能证明未披露组件不存在；未来工作应由两名独立编码者计算一致性。第三，命题中的 ϵ_i 只有在目标分布、错误定义和估计协议固定时才有数值意义；现实中的跨层错误常相关，并集界可能很松。第四，形式化内核的可信性仍依赖实现、编译器、硬件和公理基础，不能被描述为形而上的“绝对正确”。第五，语义等价本身可能需要专家判断，难以完全自动化。第六，本文未运行新模型，因而不对任何系统的实际风险降低幅度作因果声明。

9 结论

AI4Math 的核心挑战不是让模型更频繁地产生看似正确的答案，而是让每个公开结论都携带一条可追溯、可检查、可重放的证据链。本文提出的 TRACE 框架把可靠性分解为任务/数据溯源、推理搜索、语义对齐、确定性确证和评估重放五个门控，并用残余错误并集界说明端到端可靠性受最薄弱环节约束。跨研究分析表明，搜索与过程监督改善候选筛选，形式化内核强化局部正确性，公开基准支撑比较；但语义规约失真、验证器共偏差、数据污染和复现环境漂移仍不能由单一技术消除。未来工作应在同一预算下实证比较各门消融，并优先建立可审计的自然语言-形式语义对齐数据。对高风险数学主张，最诚实的系统不是从不拒答，而是清楚说明自己证明了什么、依据什么、在哪些条件下可以被再次验证。

AI 使用与研究诚信声明

本文在资料整理、框架命名、结构草拟、LaTeX 排版与参考文献格式化中使用了生成式 AI。所有进入参考文献表的条目均通过出版商、会议官方页面、Crossref 或 arXiv 标识进行来源核对；未把模型生成的摘要当作一手证据。AI 曾把 DOI 含 2025 的 AlphaProof 论文误记为 2025 年卷期论文，经 Crossref 核对后按 Nature 第 651 卷第 8106 期的印刷年份 2026 记录，并保留 online/print 差异说明。论文不声称开展了未实际执行的模型实验；表格为机制编码，非性能复现。完整 AI 协作与核验轨迹见随稿的《AI 协作日志.md》，七维复盘见《七维 AAR.md》。投稿前仍应由作者本人完成逐条语义终审并签名确认。

数据与材料可用性

本文不产生新的训练数据。论文源文件、BibTeX 数据库、文献核验表、AI 协作日志与 AAR 随稿提供。所有外部论文通过参考文献中的 DOI、官方会议页面或 arXiv URL 定位。

参考文献

- [1] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021. URL <https://arxiv.org/abs/2110.14168>.
- [2] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang,

- Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, volume 1, 2021. URL <https://datasets-benchmarks-proceedings.neurips.cc/paper/2021/hash/be83ab3ecd0db773eb2dc1b0a17836a1-Abstract-round2.html>.
- [3] Jonathan Uesato, Nate Kushman, Ramana Kumar, Francis Song, Noah Siegel, Lisa Wang, Antonia Creswell, Geoffrey Irving, and Irina Higgins. Solving math word problems with process- and outcome-based feedback. *arXiv preprint arXiv:2211.14275*, 2022. URL <https://arxiv.org/abs/2211.14275>.
- [4] Hunter Lightman, Vineet Kosaraju, Yuri Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s verify step by step. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=v8L0pN6EOi>.
- [5] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, volume 12699 of *Lecture Notes in Computer Science*, pages 625–635. Springer, 2021. doi: 10.1007/978-3-030-79876-5_37.
- [6] The mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs*, pages 367–381. Association for Computing Machinery, 2020. doi: 10.1145/3372885.3373824.
- [7] Yuhuai Wu, Albert Q. Jiang, Wenda Li, Markus N. Rabe, Charles Staats, Mateja Jamnik, and Christian Szegedy. Autoformalization with large language models. In *Advances in Neural Information Processing Systems*, volume 35, 2022. URL <https://openreview.net/forum?id=IUikebJ1Bf0>.
- [8] Zhangir Azerbayev, Bartosz Piotrowski, Hailey Schoelkopf, Edward W. Ayers, Dragomir Radev, and Jeremy Avigad. ProofNet: Autoformalizing and formally proving undergraduate-level mathematics. *arXiv preprint arXiv:2302.12433*, 2023. URL <https://arxiv.org/abs/2302.12433>.
- [9] Peiyi Wang, Lei Li, Zhihong Shao, Runxin Xu, Damai Dai, Yifei Li, Deli Chen, Yu Wu, and Zhifang Sui. Math-shepherd: Verify and reinforce LLMs step-by-step without human annotations. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 9426–9439, 2024. doi: 10.18653/v1/2024.acl-long.510. URL <https://aclanthology.org/2024.acl-long.510/>.
- [10] Aitor Lewkowycz, Anders Andreassen, David Dohan, Ethan Dyer, Henryk Michalewski, Vinay Ramasesh, Ambrose Slone, Cem Anil, Imanol Schlag, Theo Gutman-Solo, Yuhuai Wu, Behnam Neyshabur, Guy Gur-Ari, and Vedant Misra. Solving quantitative reasoning problems with language models. *Advances in Neural Information Processing Systems*, 35: 3843–3857, 2022. URL <https://arxiv.org/abs/2206.14858>.

- [11] Chaoqun He, Renjie Luo, Yuzhuo Bai, Shengding Hu, Zhen Leng Thai, Junhao Shen, Jinyi Hu, Xu Han, Yujie Huang, Yuxiang Zhang, Jie Liu, Lei Qi, Zhiyuan Liu, and Maosong Sun. OlympiadBench: A challenging benchmark for promoting AGI with olympiad-level bilingual multimodal scientific problems. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 3828–3850, 2024. doi: 10.18653/v1/2024.acl-long.211. URL <https://aclanthology.org/2024.acl-long.211/>.
- [12] Elliot Glazer, Ege Erdil, Tamay Besiroglu, Diego Chicharro, Evan Chen, Alex Gunning, et al. FrontierMath: A benchmark for evaluating advanced mathematical reasoning in AI. *arXiv preprint arXiv:2411.04872*, 2024. URL <https://arxiv.org/abs/2411.04872>.
- [13] Kunhao Zheng, Jesse Michael Han, and Stanislas Polu. MiniF2F: A cross-system benchmark for formal olympiad-level mathematics. In *The Tenth International Conference on Learning Representations*, 2022. URL <https://openreview.net/forum?id=9ZPegFuFTFv>.
- [14] Albert Q. Jiang, Sean Welleck, Jin Peng Zhou, Timothee Lacroix, Jiacheng Liu, Wenda Li, Mateja Jamnik, Guillaume Lample, and Yuhuai Wu. Draft, sketch, and prove: Guiding formal theorem provers with informal proofs. In *The Eleventh International Conference on Learning Representations*, 2023. URL <https://openreview.net/forum?id=SMa9EAovKMC>.
- [15] Kaiyu Yang, Aidan M. Swope, Alex Gu, Rahul Chalamala, Peiyang Song, Shixing Yu, Saad Godil, Ryan Prenger, and Anima Anandkumar. LeanDojo: Theorem proving with retrieval-augmented language models. In *Advances in Neural Information Processing Systems*, volume 36, 2023. URL <https://openreview.net/forum?id=g70X2s0Jtn>.
- [16] Gabriel Poesia and Noah D. Goodman. Peano: Learning formal mathematical reasoning. *Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences*, 381(2251):20220044, 2023. doi: 10.1098/rsta.2022.0044.
- [17] Trieu H. Trinh, Yuhuai Wu, Quoc V. Le, He He, and Thang Luong. Solving olympiad geometry without human demonstrations. *Nature*, 625(7995):476–482, 2024. doi: 10.1038/s41586-023-06747-5.
- [18] Bernardino Romera-Paredes, Mohammadamin Barekatin, Alexander Novikov, Matej Balog, M. Pawan Kumar, Emilien Dupont, Francisco J. R. Ruiz, Jordan S. Ellenberg, Pengming Wang, Omar Fawzi, Pushmeet Kohli, and Alhussein Fawzi. Mathematical discoveries from program search with large language models. *Nature*, 625(7995):468–475, 2024. doi: 10.1038/s41586-023-06924-6.
- [19] Thomas Hubert, Rishi Mehta, Laurent Sartran, Miklos Z. Horvath, Goran Zuzic, Eric Wieser, Aja Huang, et al. Olympiad-level formal mathematical reasoning with reinforcement learning. *Nature*, 651(8106):607–613, 2026. doi: 10.1038/s41586-025-09833-y. Published online in 2025; print issue dated 2026.

A TRACE 审计清单

门	审计问题	最低通过证据
T	原题是否完整、唯一、可追踪?	原始输入及哈希; 量词、数域、单位、图像条件; 数据来源与许可证
T	测试题是否可能被训练数据污染?	公开时间、去重方法、保密/动态题协议或污染风险声明
R	搜索是否公平比较?	同一模型与提示; 相同 token/调用预算; k 、温度、随机种子
R	验证器是否独立、校准且抗分布漂移?	独立测试集、校准曲线、对抗错误集、共享语料披露
A	形式化陈述是否与原题等价?	正反向释义; 约束对照; 边界/反例测试; 人工签核
A	隐含条件是否被显式化?	非退化、定义域、单位、近似与容差清单
C	检查是否确定、可重放?	证明对象/证书; 内核、求解器、库与版本; 完整日志
C	检查器验证的结论边界是什么?	公理、依赖定理、数值误差或程序规格说明
E	第三方能否重建实验?	依赖锁、镜像摘要、配置、脚本、随机种子、硬件说明
E	负面结果是否保留?	全部失败样本、拒答、超时、人工裁决与排除原因

B 建议的可靠性报告模板

1. **主张**: 用一句可证伪的话描述, 不使用“接近人类”等模糊措辞。
2. **任务证据**: 数据版本、样本范围、污染风险、授权与预处理。
3. **生成与搜索**: 模型、提示、解码、预算、候选保留规则。
4. **语义规约**: IR/形式语言、双向对照、人工审阅者与分歧裁决。
5. **确证**: 检查器、版本、证书、失败类型和可信计算基。
6. **评估**: 主要/次要指标、置信区间、风险-覆盖率、成本。
7. **复现**: 环境、脚本、种子、日志、已知不确定性和可访问性。